DBMS TASK

Order Management System

Aim: To design an Order Management System using Orders and Order_Details tables, store customer orders, modify order information, and generate customer order history.

1. Requirements

• Create Orders and Order_Details tables.

• Store customer and product order information.

• Store order date, quantity and total amount.

• Perform order insertion and modification.

• Generate customer order history.

2. SQL Code

```
CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Customer_ID INT,
    Order_Date DATE,
    Total_Amount DECIMAL(10,2)
);

CREATE TABLE Order_Details (
    Detail_ID INT PRIMARY KEY,
    Order_ID INT,
    Product_ID INT,
    Quantity INT,
    Price DECIMAL(10,2),
    FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)
);

-- Insert an order
INSERT INTO Orders
VALUES (101, 1, '2026-08-17', 1500.00);

INSERT INTO Order_Details
VALUES (1, 101, 501, 2, 750.00);

-- Modify order
UPDATE Orders
SET Total_Amount = 1600.00
WHERE Order_ID = 101;

UPDATE Order_Details
SET Quantity = 3
WHERE Detail_ID = 1;

-- Customer order history
SELECT o.Order_ID, o.Customer_ID, o.Order_Date,
       d.Product_ID, d.Quantity, d.Price,
       o.Total_Amount
FROM Orders o
JOIN Order_Details d
ON o.Order_ID = d.Order_ID
WHERE o.Customer_ID = 1;
```

3. Sample Output

Order ID	Customer ID	Date	Product ID	Quantity	Total
101	1	2026-08-17	501	3	1600.00

4. Operations Performed

• **Insert:** Adds a new customer order and its product details.

• **Update:** Changes the total amount or quantity of an existing order.

• **History Report:** JOIN displays the customer's order date, product, quantity and total amount.

5. Conclusion

The Order Management System stores customer orders in related tables. Orders contains general order information, while Order_Details stores product-level details. SQL INSERT, UPDATE and JOIN operations are used to manage orders and generate customer order history reports.